

Mapping MXFP to FPGAs

Marwan Mekhemer
Department of ECE
University of Waterloo
Waterloo, Ontario, Canada
msmekhemer@uwaterloo.ca

Balaji Venkatesh
Department of ECE
University of Waterloo
Waterloo, Ontario, Canada
balaji.venkatesh@uwaterloo.ca

Ahmed Elsousy
Department of ECE
University of Waterloo
Waterloo, Ontario, Canada
aelsousy@uwaterloo.ca

Abstract—Hello Professor Nachiket!
Please enjoy our preliminary paper.
Index Terms—FPGA, MXFP

I. INTRODUCTION

Modern AI models continue to grow in parameter count, requiring increased computing power and storage capacity. As such, there are numerous efforts to decrease the parameter size by using lower precision arithmetic. Originally starting from 32-bit floating point (FP32), newer models have started to use narrower formats, such as FP8 and INT8. However, in these precisions, the dynamic range is severely limited. For example, in the version of FP8 with a 4-bit exponent and a 3-bit mantissa (E4M3), the maximum representable value is 448, and the minimum representable value is 2^{-9} . In order to improve the range, these formats use tensor-level scaling factors to adjust the range for each vector. To go even smaller, MFXP formats introduce sub-tensor or block-level scaling for every 32 elements. This is called "microscaling".

In this work, we develop a benchmarking method and explore methods to map MXFP arithmetic to current FPGAs. Then we ... (we will fill this out when we get past this point).

II. BACKGROUND

A. MXFP Format Overview

The MXFP format consists of blocks with length k . Each block has a shared scale X and k scalar elements $[P_1 \dots P_k]$. The scale is multiplied to all elements in the block, resulting in the vector $[XP_1 \dots XP_k]$, as shown in Figure 1. Standardizations of this format are published in [1]. All specified formats share some parameters. The size of the block is always $k = 32$,

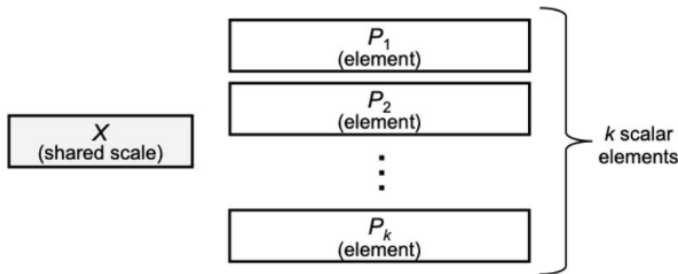


Fig. 1. Overview of the MXFP format.

and the scaling factor X is always encoded in E8M0, such that $X = 2^A$ where A is INT8. The size and format of the individual elements varies. MXFP8 takes up 8 bits and can be encoded as E5M2 or E4M3, MXFP6 takes up 6 bits and can be encoded as E3M2 or E2M3, and MXFP4 takes up 4 bits and is encoded as E2M1. The exponent bias for all these formats can be computed as $bias = 2^E/2 - 1$.

B. MXFP Special Values

All MXFP formats support subnormals. MXFP-8 E5M2 starts with encodings for infinities and NaN, but as the formats get smaller, these encodings are no longer available. Tables I, II, and III, show the special value encodings for MXFP8, MXFP6, and MXFP4 respectively.

C. Dot Products in MXFP

Given two MXFP vectors of length k , their dot product is a scalar number C calculated as follows.

$$\begin{aligned} A &= [X_A P_{A1} \dots X_A P_{Ak}] \\ B &= [X_B P_{B1} \dots X_B P_{Bk}] \\ C &= A \cdot B = X_A X_B \sum_{i=1}^k (P_{Ai} P_{Bi}) \end{aligned}$$

This is extended for a general dot product by computing a block-wise dot product along the length of longer vectors,

$$\begin{aligned} A &= [X_A P_{A1} \dots X_A P_{Ak}, X_A P_{Ak+1} \dots X_A P_{An}] \\ B &= [X_B P_{B1} \dots X_B P_{Bk}, X_B P_{Bk+1} \dots X_B P_{An}] \\ C &= A \cdot B \\ &= A_{1\dots k} \cdot B_{1\dots k} \\ &\quad + A_{k+1\dots 2k} \cdot B_{k+1\dots 2k} \\ &\quad + \dots \\ &\quad + A_{jk+1\dots n} \cdot B_{jk+1\dots n} \end{aligned}$$

III. DOT PRODUCT BENCHMARK CIRCUITS

In order to assess the current performance of implementations using MXFP on FPGAs, we consider the MXFP dot product circuit presented by [2], which allows parametrization of dot product length and exponent and mantissa bits to specify the circuit to a particular MXFP format. It is important to note

TABLE I
FP8 SPECIAL VALUES [1]

Value	E4M3	E5M2
Exponent Bias	7	15
Infinities	N/A	S 1111 00
NaN	S 1111 111	S 11111 {01, 10, 11}
Zeros	S 0000 000	S 00000 00
Max normal	S 1111 110	S 11110 11
Min normal	S 0001 000	S 00001 00
Max subnormal	S 0000 111	S 00000 11
Min subnormal	S 0000 001	S 00000 01

TABLE II
FP6 SPECIAL VALUES [1]

Value	E2M3	E3M2
Exponent Bias	1	3
Infinities	N/A	N/A
NaN	N/A	N/A
Zeros	S 00 000	S 000 00
Max normal	S 11 111	S 111 11
Min normal	S 01 000	S 001 00
Max subnormal	S 00 111	S 000 11
Min subnormal	S 00 001	S 000 01

that these circuits do not provide the final result in FP32, but rather provide an exact result in a fixed point format dependent on the MXFP format specified. This should be taken into account when comparing area and frequency results to alternative implementations.

To obtain timing results from the circuits, we make the addition of input and output register stages to the circuit, and add 5 additional register stages, with retiming enabled in Quartus. Multiplication is performed both in soft logic and DSPs (utilizing the 18x18 fixed point multiplication mode) through the use of synthesis attributes. This provides baseline area and frequency results to compare alternative implementations to.

The frequency and area results for all MXFP formats and dot product lengths of 8 and 32 are presented in Table III. We note that the ALM utilization of the circuit largely depends on both the number of exponent bits and the size of the dot product, with the number of mantissa bits playing a less significant role.

We also note that the use of synthesis attributes to automatically map multiplications to DSPs had minimal impact on ALM utilization, and required the use of one DSP for every two multiplications performed (due to the use of the 18x18 fixed point multiplication mode), indicating that multiplications with small widths (largest width is 5 for the circuits used) are not effectively mapped to DSPs through the use of synthesis attributes.

IV. CURRENT OPTIONS FOR IMPLEMENTING MXFP DOT PRODUCT

Following from section III, we consider more effective methods to utilize the existing Agilex 5 DSP block in order to implement an MXFP dot product. The DSP block is capable of operating in a number of different modes, which operate on different number formats, documented in [2]. In this section

TABLE III
FP4 SPECIAL VALUES [1]

Value	E2M1
Exponent Bias	1
Infinities	N/A
NaN	N/A
Zeros	S 00 0
Max normal	S 11 1
Min normal	S 01 0
Max subnormal	S 00 1
Min subnormal	S 00 1

we provide a brief description of the modes deemed to be the most promising targets for MXFP formats, and their planned method of utilization. A summary of the different modes and their suitability for different MXFP formats is provided in Table IV. The highlighted options are considered the most promising and warrant further investigation.

Table IV also provides the necessary size for a fixed point representation of the given MXFP formats (INT) and the necessary bits required for exact accumulation of a 32 element dot product, as discussed in [1]. These sizes limit the operational modes an MXFP format may be able to map to.

A. Tensor mode

Tensor mode is capable of performing 20 8-bit signed multiplications in a single cycle, through the use of 10 8-bit data input operands, as well as 20 8-bit operands which must be loaded into internal buffers within the DSP. The output of this mode is provided in FP32, and is adjusted by a provided shared exponent input. Although this mode provides the most multiplications of any DSP operating mode, its limited input width prohibits its use with many of the larger MXFP formats, which may not be representable in an 8-bit fixed point format. Additional hardware is also required to convert the MXFP number into a fixed point representation usable by the DSP.

B. FP32

FP32 operational modes are capable of single multiplication of two FP32 numbers, and provide a single FP32 output. The DSP is capable of operating in a multiply-accumulate (MAC) mode and several DSPs may be chained together to directly form a dot product, as shown in section 4.4 of [3]. This removes the need to perform any addition in soft logic. Additional hardware is required to convert the MXFP input to FP32, and adjust the final FP32 output by the shared exponent.

C. FP16

FP16 operational modes are similar to FP32 modes, but allow for 2 multiplications to be performed in a single DSP, and the output remains in FP32 format. A direct vector dot product structure is also possible using FP16. Additional hardware is required to convert the MXFP input to FP16, and adjust the final FP32 output by the shared exponent, as in the case of the FP32 operational modes. This mode can be considered superior to FP32 operational modes, since all MXFP formats can be represented in FP16, and this mode

TABLE IV
AREA AND FREQUENCY RESULTS FOR DOT PRODUCT CIRCUITS WITH 7 REGISTER STAGES AND RETIMING ENABLED (S INDICATES MULTIPLICATION IN SOFT LOGIC, WHILE D INDICATES IT WAS PERFORMED IN DSP BLOCKS), K INDICATES DOT PRODUCT LENGTH

Format	Logic Utilization (ALMs)				DSP Usage				Fmax (MHz)			
	K=8, S	K=32, S	K=8, D	K=32, D	K=8, S	K=32, S	K=8, D	K=32, D	K=8, S	K=32, S	K=8, D	K=32, D
E5M2	1002	3863	1004	3714	0	0	4	16	295.77	217.96	217.77	194.29
E4M3	653	2645	752	2853	0	0	4	16	521.92	312.99	214.50	208.64
E3M2	449	1588	429	1554	0	0	4	16	850.34	332.01	222.07	214.59
E2M3	367	1430	335	1305	0	0	4	16	850.34	445.43	249.88	224.47
E2M1	235	768	233	889	0	0	4	16	850.34	850.34	282.09	255.95

TABLE V
MXFP FORMAT MAPPING OPTIONS TARGETING AN AGILEX 5 DSP

Format	INT	b_0	DSP Mode				
			Tensor	FP32	FP16	Ind. Mult. (Frac)	Mult. Add
E5M2	34	73	N/A	1 mult., FP32 accum	2 mult., FP32 accum	6 mult.	N/A
E4M3	19	43	N/A	1 mult., FP32 accum	2 mult., FP32 accum	4 mult.	1 mult., 54 bit accum
E3M2	10	25	N/A	1 mult., FP32 accum	2 mult., FP32 accum	6 mult.	2 mult., 36 bit accum
E2M3	7	19	20 mult., FP32 accum	1 mult., FP32 accum	textbf2 mult., FP32 accum	4 mult.	2 mult., 36 bit accum
E2M1	5	15	20 mult., FP32 accum	1 mult., FP32 accum	2 mult., FP32 accum	8 mult.	2 mult., 36 bit accum

TABLE VI
NUMBER OF OPERANDS ABLE TO BE PACKED INTO 27x27 AND 18x19
FIXED POINT MULTIPLICATION MODES

Format	27x27	2 18x18/19
E5M2	4	6
E4M3	3	4
E3M2	4	6
E2M3	3	4
E2M1	6	8

provides twice as many multiplications per DSP as the FP32 modes.

D. Packed fixed point multiplication

The DSP provides three options for independent fixed point multiplication, a single 27x27 (signed or unsigned) bit multiplication, two 18x18 unsigned multiplications and two 18x19 fixed point signed multiplications. While these modes can be used by converting the MXFP formats to a fixed point representation, this is analogous to the use of the floating point modes previously discussed, and would require more hardware to convert to fixed point and convert the resulting fixed point output back to FP32 and adjust the exponent. Instead, we consider packing several mantissa into a single operand of a multiplier, and multiply them by a single common multiplicand, which would be the mantissa of another MXFP number. The mantissa packed into the multiplier must be spaced out such that their results do not overlap. The number of multiplications able to be packed in a single DSP for the different MXFP formats is shown in Table VI. This approach requires that the multiplication results be shifted according to the individual MXFP number's exponents, and summed in soft logic. Additionally, the result of this sum must be converted to FP32 and the shared exponent must be used to adjust the final exponent.

V. RESULTS AND DISCUSSION

In order to verify correct operation of MXFP dot product circuits, a python script was developed to emulate the operations which are expected to be performed in hardware. The main function of the script is to generate random MXFP vectors of a given format and provide the fixed point and FP32 results of the dot product of those vectors. The size of exponent and mantissa fields, the length of the generated vectors and the number of test vectors, are provided as arguments to the script. Afterwards, a simple testbench can then compare the output of the current design with the expected output from the script in order to verify the design's functionality. The generated data was used to verify the functionality of all benchmark circuits presented in section III, and for any alternative designs.

A. AI Tensor Mode Dot Product

As seen in Table IV, the AI Tensor mode is a great candidate for MX-FP4 and MX-FP6(E2M3) dot products as they both fit in an INT8 fixed point representation, where MX-FP4 is converted to $sfix(7, -1)$ and MX-FP6 (E2M3) is converted to $sfix(5, -3)$. A floating point to fixed point conversion circuit was implemented according to equation 1 for normal MX-FP numbers and 2 for subnormals.

$$FXP = ((1 \ll M) + M) \ll (E - E_{bias}) \quad (1)$$

$$FXP = M \ll (E - E_{bias}) \quad (2)$$

We will use the AI Tensor mode of the DSP in floating point mode, which outputs the 10-way dot product in single-precision FP32 and allows an 8-bit shared exponent or scale input in the form of E8M0 with a 127 bias, which perfectly matches the MX-FP scale term specification. Thus, to support the block size of the MX-FP specification (32 elements) [1], 4 DSPs are chained together as per the documentation of it

TABLE VII
MX-FP 32-DOT PRODUCT RESULTS IN AI TENSOR MODE
IMPLEMENTATION

Format	MX-FP4 (E2M1)	MX-FP6 (E2M3)
ALMs	135	250
FFs	148	188
DSPs	4	4
F_{max} (MHz)	457.67	457.67

[3]. This creates a small inefficiency as the fourth DSP has only 2 inputs populated with actual inputs and the rest are set to zeros. However, this is acceptable as in a real workload as we can find other related elements that can be used instead of the zeros. For the sake of this implementation, we are leaving the zeros in to abide by the specification. Table VII shows the area and performance results of a 32-dot module for MX-FP4 and MX-FP6 (E2M3) that uses the AI Tensor in floating point mode, which outputs the dot product in FP32 as mentioned before, saving any FP32 conversion circuitry for the output. The ALM and register are mainly utilized by the floating-point to fixed-point conversion circuitry and they have a small resource footprint as expected. By far, the AI Tensor mode has the greatest arithmetic density of all mentioned approaches and the most efficient in terms of ALM utilization and DSP usage. Compared to the inferred DSP implementation seen in Table IV, the AI Tensor Block implementation is faster and uses less resources overall, winning on all regards. The other close contender is the full soft logic implementation of the dot engine as it has the highest throughput. The main caveat of using the AI Tensor mode is that it has strict data timing modes and requirements. The data loading mode is “data-feed in” in which a set of 10 INT8 vectors is loaded to registers in the DSP block then computation is carried on by doing the dot product between the input vector and the stored one. The other mode is “side-feed in” which allows the user to load the internal registers with data element by element while computation is carried on. For this implementation, we used data-feed in as it makes the most sense for workloads that benefit from low-precision floating point arithmetic in the first place, i.e., machine learning, which have some operands that do not change frequently. In short, the AI Tensor mode is best used for an MX-FP4 or MX-FP6 (E2M3) dot engine.

B. FP16 Direct Vector Dot Product

An MXFP dot product circuit was implemented using the FP16 vector one, vector two and sum of two operational modes, as discussed in section 4 C, and in section 4.4 of [3]. In addition to the chained DSP blocks, which provide the result of the dot product of two MXFP vectors, additional circuits are required to convert the MXFP input the required FP16 input of the DSP blocks. While converting MXFP numbers to higher precision FP formats is trivial for normal numbers, requiring only a bias correction for the exponent, subnormal numbers require greater effort. Subnormal numbers in an MXFP format may be represented as normal numbers in FP16, and as such we must shift the MXFP mantissa such that the leading 1 is

absorbed into the implied one of the resulting FP16 number. The exponent of the FP16 number must also be adjusted to account for this correction. It is also notable that the additional hardware required for handling subnormals is only necessary if the source and target formats have unequal exponent widths. For floating point formats with equal exponent widths, subnormal numbers are represented in the same manner regardless of mantissa width. After obtaining the FP16 vectors, they are provided to the chained DSP structure described in [3], and the resulting FP32 output must be adjusted by the shared exponents of both MXFP input vectors. The described circuit is currently only verified for a length 2 dot product, and further work is required to correctly implement the design for a variable length dot product. However, area and timing results are still provided for a 32 element dot product in Table VIII using the described implementation. The resulting area and timing results, and trends observed from them, should closely resemble a fully functional variable length dot product design. As expected, all MXFP formats use 16 DSP blocks, as a result of the 2 FP16 multiplications present in each DSP block. ALM usage is dramatically lower than the benchmark circuits in Table IV, as a result of the addition being performed within the DSP as opposed to soft logic. The reported frequency values were restricted by the DSP configuration, however we may still note that the frequency is significantly improved from the inferred multiplications presented in Table III. Finally, we also note that area and frequency are not affected significantly by the choice of input MXFP format, since arithmetic operations are always performed in FP16 within the DSP blocks.

TABLE VIII
FP16 DIRECT VECTOR DOT PRODUCT AREA AND FREQUENCY RESULTS,
32 ELEMENT DOT PRODUCT

Format	Fmax	ALMs	DSPs
E5M2	459.98	201	16
E4M3	459.98	321	16
E3M2	459.98	205	16
E2M3	459.98	230	16
E2M1	459.98	138	16

REFERENCES

- [1] Open Compute Project, “OCP Microscaling Formats (MX) Specification.” <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>, 2023. Final specification.
- [2] E. Samson, N. Mellempudi, W. Luk, and G. A. Constantinides, “Exploring FPGA designs for MX and beyond,” in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*, (Los Alamitos, CA, USA), pp. 304–310, IEEE Computer Society, Sept. 2024.
- [3] Intel Corporation, “Agilex™ 5 Variable Precision DSP Blocks Overview.” <https://www.intel.com/content/www/us/en/docs/programmable/813968/25-1/variable-precision-dsp-blocks-overview.html>, Aug. 2025. Version 25.1.